

Problem Set 1

2025-10-29

Burak Çizmecioğlu 34375

Libraries

```
# install the pokemon library if you dont  
library(pokemon)  
# Load the dataset  
data("pokemon")  
# Quick view  
head(pokemon)
```

```
## id    pokemon species_id height weight base_experience type_1 type_2 hp
## 1 1    bulbasaur      1    0.7   6.9             64 grass poison 45
## 2 2    ivysaur       2    1.0  13.0            142 grass poison 60
## 3 3    venusaur      3    2.0 100.0            236 grass poison 80
## 4 4    charmander    4    0.6   8.5             62  fire  <NA> 39
## 5 5    charmeleon    5    1.1  19.0            142  fire  <NA> 58
## 6 6    charizard     6    1.7  90.5            240  fire flying 78
##   attack defense special_attack special_defense speed color_1 color_2 color_f
## 1     49      49              65              65   45 #78C850 #A040A0 #81A763
## 2     62      63              80              80   60 #78C850 #A040A0 #81A763
## 3     82      83             100             100   80 #78C850 #A040A0 #81A763
## 4     52      43              60              50   65 #F08030  <NA>   <NA>
## 5     64      58              80              65   80 #F08030  <NA>   <NA>
## 6     84      78             109              85  100 #F08030 #A890F0 #DE835E
##   egg_group_1 egg_group_2
## 1    monster    plant
## 2    monster    plant
## 3    monster    plant
## 4    monster    dragon
## 5    monster    dragon
## 6    monster    dragon
##                                     url_icon generation_id
## 1 //archives.bulbagarden.net/media/upload/7/7b/001MS6.png           1
## 2 //archives.bulbagarden.net/media/upload/a/a0/002MS6.png           1
## 3 //archives.bulbagarden.net/media/upload/0/07/003MS6.png           1
## 4 //archives.bulbagarden.net/media/upload/7/7d/004MS6.png           1
## 5 //archives.bulbagarden.net/media/upload/b/be/005MS6.png           1
## 6 //archives.bulbagarden.net/media/upload/4/41/006MS6.png           1
##                                     url_im
age
## 1 https://raw.githubusercontent.com/HybridShivam/Pokemon/master/assets/images/001.
png
## 2 https://raw.githubusercontent.com/HybridShivam/Pokemon/master/assets/images/002.
png
## 3 https://raw.githubusercontent.com/HybridShivam/Pokemon/master/assets/images/003.
png
## 4 https://raw.githubusercontent.com/HybridShivam/Pokemon/master/assets/images/004.
png
## 5 https://raw.githubusercontent.com/HybridShivam/Pokemon/master/assets/images/005.
png
## 6 https://raw.githubusercontent.com/HybridShivam/Pokemon/master/assets/images/006.
png
```

```
# Essential libraries
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
## intersect, setdiff, setequal, union
```

```
library(tidyr)  
library(ggplot2)
```

Question 1 (5 points)

Which type is the most common in the dataset? Note that each Pokémon can have up to two types (type_1 and type_2). Show your code.

```
# Select type_1 and type_2 columns and filter out NA values  
most_common <- pokemon %>%  
  select(type_1, type_2) %>%  
  filter(!is.na(type_1) & !is.na(type_2))  
  
head(most_common)
```

```
## # A tibble: 6 × 2  
##   type_1 type_2  
##   <chr> <chr>  
## 1 grass  poison  
## 2 grass  poison  
## 3 grass  poison  
## 4 fire   flying  
## 5 bug    flying  
## 6 bug    poison
```

```
# pivot_longer to convert wide format to long format and create new columns with regard to new names  
most_common_long <- most_common %>%  
  pivot_longer(cols = c(type_1, type_2),  
               names_to = "type_name",  
               values_to = "type_value")  
  
head(most_common_long)
```

```
## # A tibble: 6 × 2  
##   type_name type_value  
##   <chr>      <chr>  
## 1 type_1    grass  
## 2 type_2    poison  
## 3 type_1    grass  
## 4 type_2    poison  
## 5 type_1    grass  
## 6 type_2    poison
```

```
# Count the numeric column and (sort=TRUE) will sort in descending order if I write s
ort=FALSE it will sort in ascending order or blank is also ascending
value <- most_common_long %>%
  count(type_value, sort = TRUE)
head(value)
```

```
## # A tibble: 6 × 2
##   type_value      n
##   <chr>      <int>
## 1 flying      123
## 2 water        78
## 3 grass        71
## 4 rock         67
## 5 bug          66
## 6 ground       61
```

```
# Get the most common type slice(1) will give you the first element
max_value <- value %>%
  slice(1)
head(max_value)
```

```
## # A tibble: 1 × 2
##   type_value      n
##   <chr>      <int>
## 1 flying      123
```

Question 2 (10 points):

Count how many Pokémon there are for each type. Show your code. Task: Create a table showing how many Pokémon belong to each type (regardless of whether it's type_1 or type_2). Hint = Use the function `pivot_longer` to have a single column with the type information.

```
# Select type_1 and type_2 columns and filter out NA values
most_common <- pokemon %>%
  select(type_1 , type_2) %>%
  filter(!is.na(type_1) & !is.na(type_2))

head(most_common)
```

```
## # A tibble: 6 × 2
##   type_1 type_2
##   <chr> <chr>
## 1 grass  poison
## 2 grass  poison
## 3 grass  poison
## 4 fire   flying
## 5 bug    flying
## 6 bug    poison
```

```
# pivot_longer to convert wide format to long format and create new columns with rega
rd to new names
most_common_long <- most_common %>%
  pivot_longer(cols = c(type_1, type_2),
               names_to = "type_name",
               values_to = "type_value")

head(most_common_long)
```

```
## # A tibble: 6 × 2
##   type_name type_value
##   <chr>      <chr>
## 1 type_1    grass
## 2 type_2    poison
## 3 type_1    grass
## 4 type_2    poison
## 5 type_1    grass
## 6 type_2    poison
```

```
# Count the numeric column and (sort=TRUE) will sort in descending order if I write s
ort=FALSE it will sort in ascending order or blank is also ascending
value <- most_common_long %>%
  count(type_value, sort = TRUE)
head(value)
```

```
## # A tibble: 6 × 2
##   type_value     n
##   <chr>      <int>
## 1 flying      123
## 2 water        78
## 3 grass        71
## 4 rock         67
## 5 bug          66
## 6 ground       61
```

```
#IMPORTANT NOTES:
# value will give the entire data
# head(value) will give you the first 6 rows to easiness
```

Question 3 (5 points):

Which Pokémon is the heaviest? Which Pokémon is the tallest/shortest? Show your code.

```
weight_column <- pokemon %>%
  select(pokemon, weight)
head(weight_column)
```

```
## # A tibble: 6 × 2
##   pokemon    weight
##   <chr>      <dbl>
## 1 bulbasaur    6.9
## 2 ivysaur     13
## 3 venusaur   100
## 4 charmander   8.5
## 5 charmeleon  19
## 6 charizard   90.5
```

```
# Using == is critical because we are assigning smt, not defining a subject
heaviest <- weight_column %>%
  filter(weight == max(weight))
head(heaviest)
```

```
## # A tibble: 2 × 2
##   pokemon    weight
##   <chr>      <dbl>
## 1 cosmoem   1000.
## 2 celesteela 1000.
```

```
height_column <-pokemon %>%
  select(pokemon,height)
head(height_column)
```

```
## # A tibble: 6 × 2
##   pokemon    height
##   <chr>      <dbl>
## 1 bulbasaur    0.7
## 2 ivysaur      1
## 3 venusaur     2
## 4 charmander   0.6
## 5 charmeleon   1.1
## 6 charizard    1.7
```

```
tallest <- height_column %>%
  filter(height == max(height))
head(tallest)
```

```
## # A tibble: 1 × 2
##   pokemon height
##   <chr>      <dbl>
## 1 wailord   14.5
```

```
shortest <- height_column %>%
  filter(height == min(height))
head(shortest)
```

```
## # A tibble: 5 × 2
##   pokemon height
##   <chr>      <dbl>
## 1 joltik      0.1
## 2 flabebe    0.1
## 3 cutiefly   0.1
## 4 comfey     0.1
## 5 cosmoem    0.1
```

Question 4 (5 points):

Which generation has the most Pokémon? Find out how many Pokémon belong to each generation. Then visualize it using a bar plot. Show your code.

```
# Wanted columns and filtering NAs
generation_column <- pokemon %>%
  select(pokemon, generation_id) %>%
  filter(!is.na(generation_id))
head(generation_column)
```

```
## # A tibble: 6 × 2
##   pokemon generation_id
##   <chr>          <dbl>
## 1 bulbasaur        1
## 2 ivysaur          1
## 3 venusaur         1
## 4 charmander       1
## 5 charmeleon       1
## 6 charizard        1
```

```
#generation counts by grouping generation_id
generation_counts <- generation_column %>%
  group_by(generation_id) %>% # Bundan sonra yapacağım tüm hesaplamaları, 1. Nesil,
  # 2. Nesil, 3. Nesil... gibi gruplar için ayrı ayrı yap.
  summarise(count = n()) %>% # count = n(): Bu sayım sonucunu count adında yeni bir s
  # ütuna atar
  arrange(generation_id) # no need to asc() because it is defaulty ascending
head(generation_counts)
```

```
## # A tibble: 6 × 2
##   generation_id count
##   <dbl> <int>
## 1      1    151
## 2      2    100
## 3      3    135
## 4      4    107
## 5      5    156
## 6      6     72
```

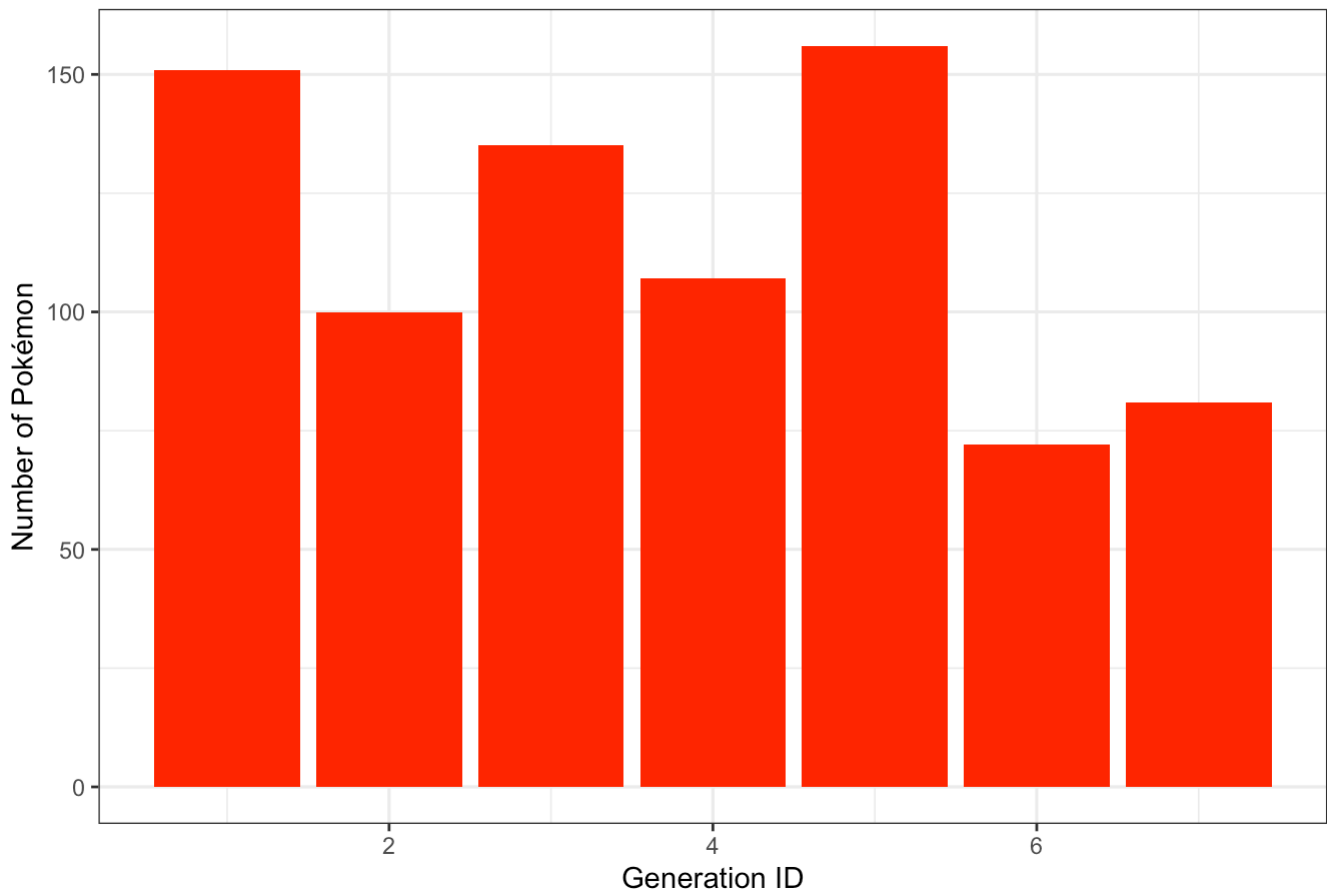
```
# First I have to filter to find the relative rows then select the generation_id column
max_generation <- generation_counts %>%
  filter(count == max(count)) %>%
  select(generation_id)
head(max_generation)
```

```
## # A tibble: 1 × 1
##   generation_id
##           <dbl>
## 1             5
```

```
# Visualization part
```

```
#stat="identity": Use the actual values in the data for the heights of the bars
#stat="count": Count the number of occurrences of each x value and use that for the heights of the bars
# we already calculated the counts, so we use stat="identity"
visual <- ggplot(generation_counts, aes(x=generation_id, y=count)) +
  geom_bar(stat="identity", fill="red") +
  labs(title="Number of Pokémon by Generation",
       x="Generation ID",
       y="Number of Pokémon") +
  theme_bw()
visual
```

Number of Pokémon by Generation



Question 5 (10 points):

What is the strongest Pokémon? Task: Create a new column `total_stats` = sum of all base stats (hp, attack, defense, sp_attack, sp_defense, speed). Show the top 10 Pokémon by total stats

```
power <- pokemon %>%
  mutate(total_stats = hp+attack+defense+special_attack+special_defense+speed) %>%
  arrange(desc(total_stats)) %>%
  slice(1:10)
power
```

```
## # A tibble: 10 × 23
##       id pokemon      species_id height weight base_experience type_1 type_2    hp
##   <dbl> <chr>          <dbl>   <dbl>  <dbl>         <dbl> <chr>  <chr> <dbl>
## 1 10043 mewtwo-me...      150     2.3   127           351 psych... fight... 106
## 2 10044 mewtwo-me...      150     1.5    33           351 psych... <NA>    106
## 3 10079 rayquaza-...      384    10.8   392           351 dragon flying 105
## 4 10077 kyogre-pr...      382     9.8   430           347 water  <NA>    100
## 5 10078 groudon-p...      383     5     1000.         347 ground fire 100
## 6 493 arceus          493     3.2   320           324 normal <NA>    120
## 7 10120 zygarde-c...      718     4.5   610           319 dragon ground 216
## 8 10022 kyurem-bl...      646     3.3   325           315 dragon ice 125
## 9 10023 kyurem-wh...      646     3.6   325           315 dragon ice 125
## 10 10049 tyranitar...      248     2.5   255           315 rock  dark 100
## # i 14 more variables: attack <dbl>, defense <dbl>, special_attack <dbl>,
## #   special_defense <dbl>, speed <dbl>, color_1 <chr>, color_2 <chr>,
## #   color_f <chr>, egg_group_1 <chr>, egg_group_2 <chr>, url_icon <chr>,
## #   generation_id <dbl>, url_image <chr>, total_stats <dbl>
```

```
# First use the slice to show the related columns in the function, eventually use power
#head() is for the first 6 rows
```

Question 6 (5 points):

Now print the picture of one of the tallest or heaviest Pokémon here. Note that there is a column in the data that includes links for photos of each Pokémon. Show your code.

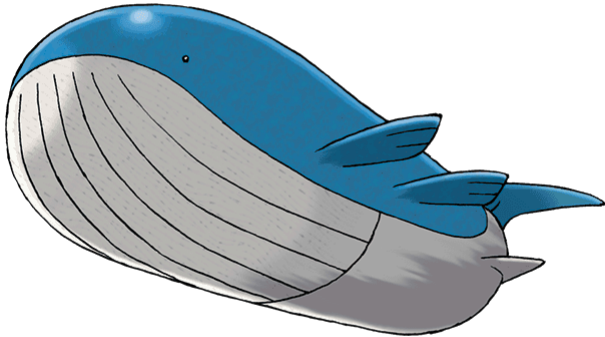
```
library(magick) # For image processing
```

```
## Linking to ImageMagick 6.9.13.29
## Enabled features: cairo, fontconfig, freetype, heic, lcms, pango, raw, rsvg, webp
## Disabled features: fftw, ghostscript, x11
```

```
tallest_pok <- pokemon %>%
  select(pokemon,height,url_image) %>%
  filter(height == max(height))
head(tallest_pok)
```

```
## # A tibble: 1 × 3
##   pokemon height url_image
##   <chr>     <dbl> <chr>
## 1 wailord  14.5 https://raw.githubusercontent.com/HybridShivam/Pokemon/master/...
```

```
image_link <- tallest_pok$url_image
image <- image_read(image_link) # View the image
image
```



#IMPORTANT NOTES:

```
# image for entire visual
# head(image) preview - act same as image in that situation
# Image is seen on local R, but not in PDF
```

Question 7 (10 points):

Now, visualize the relationship between attack and defense. What is your conclusion? Show your code. Task: Create a scatter plot of attack vs defense, colored by type_1.

- Extra: I will show my code for remembering

```
a_d <- pokemon %>%
  select(attack, defense, type_1)
head(a_d)
```

```
## # A tibble: 6 × 3
##   attack defense type_1
##   <dbl>   <dbl> <chr>
## 1     49     49 grass
## 2     62     63 grass
## 3     82     83 grass
## 4     52     43 fire
## 5     64     58 fire
## 6     84     78 fire
```

```
scatter_plot <- ggplot(a_d, aes(x=attack, y=defense, color=type_1))+
  geom_point()+ # scattering is made by geom_point()
  labs(title="Attack vs Defense",
        x="Attack",
        y="Defense")+
  theme_bw()
scatter_plot
```



color (Estetik): Grafikteki noktaların görsel özelliğidir (nokta rengi).

type_1 (Değişken): a_d veri çerçevenizdeki, Pokémon'un birincil tipini (örneğin: 'Grass', 'Fire', 'Water') belirten kategorik bir sütundur.

#ggplot2, bu işlemeyi gördüğünde otomatik olarak şu işlemi yapar: type_1 sütunundaki her bir benzersiz değere (her bir Pokémon tipine) otomatik olarak farklı bir renk atar.

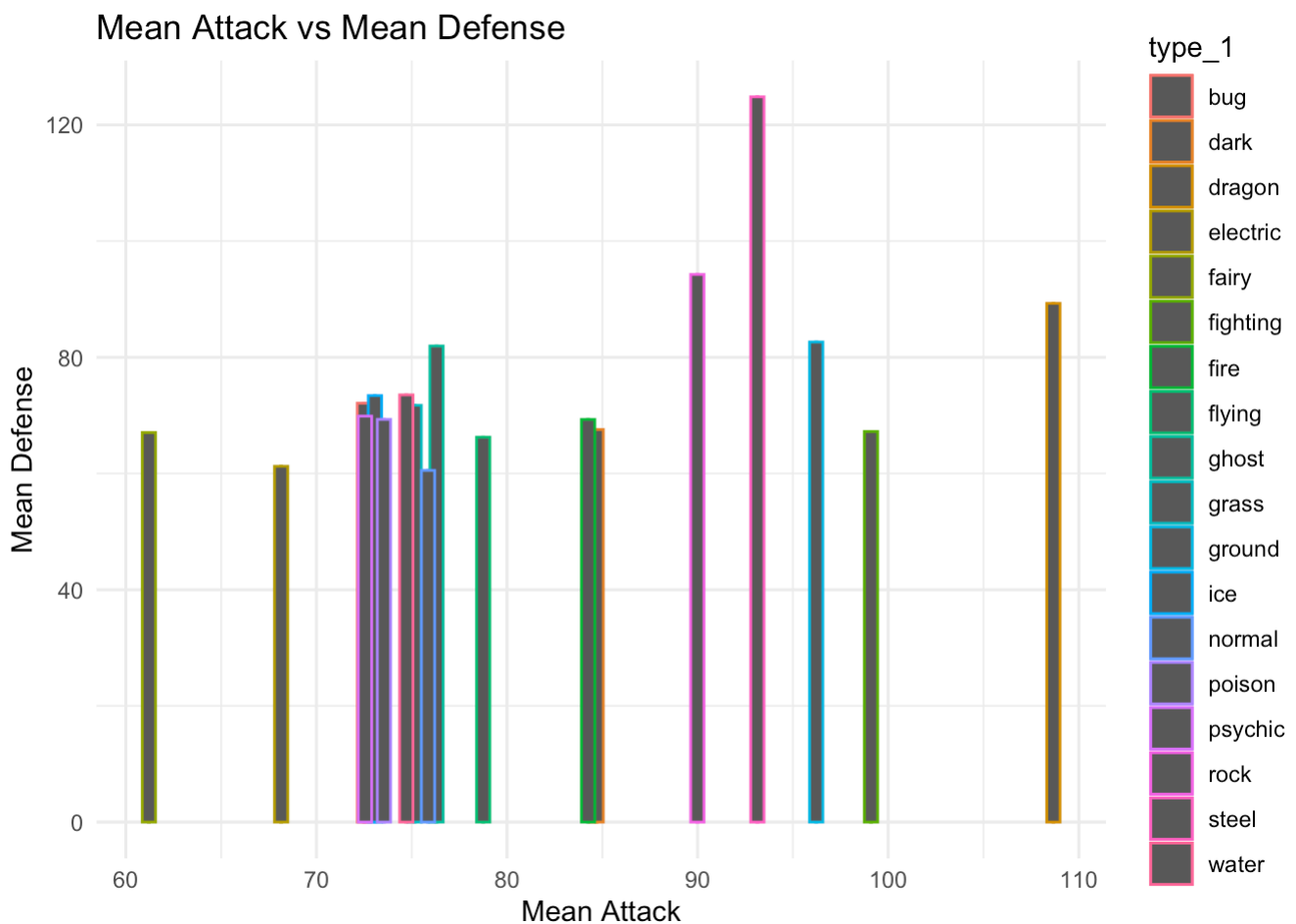
Question 8 (10 points):

Now, please compare average stats across primary types. Show your code. Task: For each type_1, calculate the average attack and average defense. Plot them as side-by-side bars.

```
compare <- pokemon %>%
  group_by(type_1) %>% # if it says for each type_1, we will use group_by func
  summarise(mean_attack = mean(attack),
            mean_defense = mean(defense))
#summarise will give a new data-frame according to desire
#if we do the same with mutate, it will add extra 2 columns to original pokemon data set
head(compare)
```

```
## # A tibble: 6 × 3
##   type_1    mean_attack mean_defense
##   <chr>      <dbl>      <dbl>
## 1 bug         72.5        72.1
## 2 dark        84.7        67.6
## 3 dragon      109.        89.3
## 4 electric    68.1        61.2
## 5 fairy       61.2        67.1
## 6 fighting    99.1        67.2
```

```
plot_compare <- ggplot(compare, aes(x=mean_attack, y=mean_defense,color=type_1))+
  geom_bar(stat="identity")+
  labs(title="Mean Attack vs Mean Defense",
       x="Mean Attack",
       y="Mean Defense")+
  theme_minimal()+
  geom_bar(width = 0.7, stat="identity")
plot_compare
```



```
# used color=type_1 to differentiate the bars by type_1
# geom_col() is an alternative to geom_bar(stat="identity")
```

Question 9 (10 points):

Now, analyze the relationship between speed and total stats. Task: Create a scatter plot of speed vs total_stats.

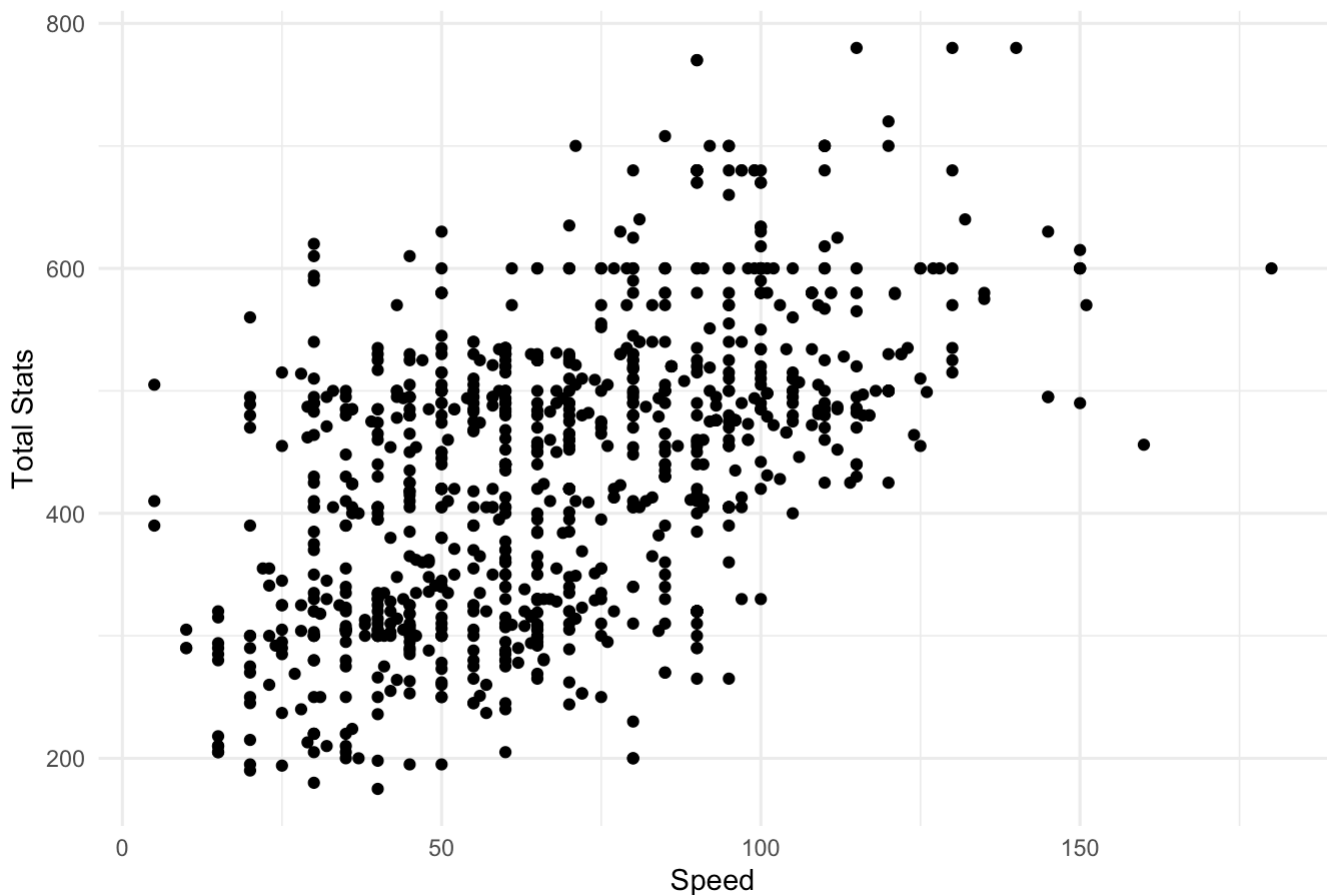
- Extra: I will show my code for remembering

```
power2 <- pokemon %>%
  mutate(total_stats = hp+attack+defense+special_attack+special_defense+speed) %>%
  arrange(desc(total_stats))
head(power2)
```

```
## # A tibble: 6 × 23
##   id pokemon      species_id height weight base_experience type_1 type_2    hp
##   <dbl> <chr>          <dbl> <dbl> <dbl>         <dbl> <chr> <chr> <dbl>
## 1 10043 mewtwo-meg...      150   2.3  127          351 psych... fight...  106
## 2 10044 mewtwo-meg...      150   1.5   33          351 psych... <NA>    106
## 3 10079 rayquaza-m...      384  10.8  392          351 dragon flying  105
## 4 10077 kyogre-pri...      382   9.8  430          347 water <NA>    100
## 5 10078 groudon-pr...      383   5    1000.        347 ground fire  100
## 6   493 arceus         493   3.2  320          324 normal <NA>    120
## # i 14 more variables: attack <dbl>, defense <dbl>, special_attack <dbl>,
## #   special_defense <dbl>, speed <dbl>, color_1 <chr>, color_2 <chr>,
## #   color_f <chr>, egg_group_1 <chr>, egg_group_2 <chr>, url_icon <chr>,
## #   generation_id <dbl>, url_image <chr>, total_stats <dbl>
```

```
speed_vs_total <- ggplot(power2, aes(x=speed, y=total_stats))+
  geom_point()+
  labs(title="Speed vs Total Stats",
       x="Speed",
       y="Total Stats")+
  theme_minimal()
speed_vs_total
```

Speed vs Total Stats



```
#Scattering is made by geom_point()
```

Question 10 (30 points): Now, simulate a one-on-one fight between two Pokémon using their Attack and HP stats. Show your code. Rules: • Each Pokémon starts with its full HP. • On each round, both Pokémon attack once. • Damage dealt by a Pokémon in a round = its Attack × a random multiplier between 0.2 and 0.6 (to add randomness). • The first Pokémon whose HP drops below zero loses. • Simulate 1000 fights for a couple of Pokémon you choose and estimate the probability that Pokémon A wins. • You can choose any two Pokémon — for example, Charizard and Blastoise. Hint: since you will simulate 1000 fights, write a function or a loop to make things easier.

```
library(stats) # for runif()
```

```
selected <- pokemon %>%
  filter(pokemon == "pikachu" | pokemon == "bulbasaur")
selected
```

```
## # A tibble: 2 × 22
##   id pokemon species_id height weight base_experience type_1 type_2 hp
##   <dbl> <chr>      <dbl> <dbl> <dbl>      <dbl> <chr> <chr> <dbl>
## 1     1 bulbasaur         1  0.7   6.9          64 grass  poison  45
## 2    25 pikachu         25  0.4    6          112 electric <NA>   35
## # i 13 more variables: attack <dbl>, defense <dbl>, special_attack <dbl>,
## #   special_defense <dbl>, speed <dbl>, color_1 <chr>, color_2 <chr>,
## #   color_f <chr>, egg_group_1 <chr>, egg_group_2 <chr>, url_icon <chr>,
## #   generation_id <dbl>, url_image <chr>
```

```

# Create a function to simulate a fight

num_simulations <- 1000

# for the renewability/flexibility of the code I will attach the pokemons first
#but the function will stay as an universal entity, so as to can be used to other pok
emons
bulbasaur_hp <- 45 # a
pikachu_hp <- 35 # b
bulbasaur_attack <- 49
pikachu_attack <- 55

# In R, we define a function using the function() command and curly braces {}

#First we need to set a general function
simulate_single <- function(hp_a, hp_b, attack_a, attack_b) { #assigning the parameters
  current_hp_a <- hp_a
  current_hp_b <- hp_b
  while(TRUE) { #Controls the loop/set infinite loop/return() will stop it
    # Pokemon A attacks Pokemon B
    damage_to_b <- attack_a * runif(1, 0.2, 0.6) #runif() for generating random
value
    current_hp_b <- current_hp_b - damage_to_b
    if (current_hp_b <= 0) {
      return("Bulbasaur") #return will give the output
    }
    # Pokemon B attacks Pokemon A
    damage_to_a <- attack_b * runif(1, 0.2, 0.6)
    current_hp_a <- current_hp_a - damage_to_a
    if (current_hp_a <= 0) {
      return("Pikachu")
    }
  }
}

#Now we have to run the simulations for 1000 times with using for loop

results <- c() #First we have to set empty vector to store results, because it will
overuse memory if we do not preallocate

for (i in 1:1000) { # 1 ve 1000 arası tüm i değerleri için for döngüsü çalışacak
  results[i] <- simulate_single ( # [i], assign a defined variable for ith index of e
ach i value
    hp_a = bulbasaur_hp,
    hp_b = pikachu_hp,
    attack_a = bulbasaur_attack,
    attack_b = pikachu_attack
  )
}

# Now we can analyze the probabilities

#We can both use sum()/1000 and mean() functions to find the probabilities
prob_bulbasaur_wins <- sum(results == "Bulbasaur") /1000
prob_pikachu_wins <- mean(results == "Pikachu")

```

```
prob_bulbasaur_wins # Estimate the probability that Pokémon A wins
```

```
## [1] 0.855
```